coursera | Johns Hopkins University

Algorithms for DNA Sequencing

Today's goal • 3/3 items

Module 1
DNA sequencing, strings and matching

Welcome

Welcome to Algorithms for DNA Sequencing
Reading • 10 min

Pre Course Survey
Reading • 10 min

Syllabus
Reading • 10 min

Setting up Python (and Jupyter)
Reading • 10 min

Getting slides and notebooks
Reading • 10 min

Using data files with Python programs
Reading • 10 min

Module 1: DNA sequencing, strings and matching

Module 1 Introduction
Video • 1 min

Lecture: Why study this?
Video • 4 min

Lecture: DNA sequencing past and present
Video • 3 min

Lecture: Genomes as strings, reads as substrings
Video • 5 min

Lecture: String definitions and Python examples
Video • 3 min

Practical: String basics
Video • 7 min

Practical: Manipulating DNA strings
Video • 7 min

Practical: Downloading and parsing a genome
Video • 6 min

Lecture: How DNA gets copied
Video • 3 min

Optional lecture: How second-generation sequencers work
Video • 7 min

Optional lecture: Sequencing errors and base qualities
Video • 6 min

Lecture: Sequencing reads in FASTQ format
Video • 4 min

Practical: Working with sequencing reads
Video • 11 min

Practical: Analyzing reads by position
Video • 6 min

Lecture: Sequencers give pieces to genomic puzzles
Video • 5 min

Lecture: Read alignment and why it's hard
Video • 3 min

Lecture: Naive exact matching
Video • 10 min

Practical: Matching artificial reads
Video • 6 min

Practical: Matching real reads
Video • 7 min

Quiz

Module 1
Graded Assignment • Grade: 100%

Programming Homework

Programming Homework 1 Instructions (Read First)
Reading • 10 min

Programming Homework 1
Graded Assignment • 30 min

Module 2
Preprocessing, indexing and approximate matching

Module 3
Edit distance, assembly, overlaps

Module 4
Algorithms for assembly

Programming Homework 1 Instructions (Read First)

0:20 / 1:26 2x

In lecture and in a practical, we saw an implementation of the naive exact matching algorithm:

```
def naive(p, t):
    occurrences = []
    for i in range(len(t) - len(p) + 1):  # loop over alignments
        match = True
        for j in range(len(p)):  # loop over characters
            if t[i+j] != p[j]:  # compare characters
                match = False
                break
        if match:
            occurrences.append(i)  # all chars matched; record
    return occurrences

```

...and we saw a function that takes a DNA string and returns its reverse complement:

```
def reverseComplement(s):
    complement = {'A': 'T', 'C': 'G', 'G': 'C', 'T': 'A', 'N': 'N'}
    t = ''
    for base in s:
        t = complement[base] + t
    return t

```

...and we saw a function that parses a DNA reference genome from a file in the FASTA format.

```
def readGenome(filename):
    genome = ''
    with open(filename, 'r') as f:
        for line in f:
            # ignore header line with genome information
            if not line[0] == '>':
                genome += line.rstrip()
    return genome

```

...and we saw a function that parses the read and quality strings from a FASTQ file containing sequencing reads.

```
def readFastq(filename):
    sequences = []
    qualities = []
    with open(filename) as fh:
        while True:
            fh.readline()  # skip name line
            seq = fh.readline().rstrip()  # read base sequence
            fh.readline()  # skip placeholder line
            qual = fh.readline().rstrip() # base quality line
            if len(seq) == 0:
                break
            sequences.append(seq)
            qualities.append(qual)
    return sequences, qualities

```

First, implement a version of the naive exact matching algorithm that is *strand-aware*. That is, instead of looking only for occurrences of P in T, additionally look for occurrences of the*reverse complement* of P in T. If P is ACT, your function should find occurrences of both ACTand its reverse complement AGT in T.

If P and its reverse complement are identical (e.g. AACGTT), then a given match offset should be reported only once. So if your new function is called naive_with_rc, then the old naivefunction and your new naive_with_rc function should return the same results when P equals its reverse complement.

Hint: See this notebook for a few examples you can use to test your naive_with_rc function.

Next, download and parse the lambda virus genome, at:
https://d28rh4a8wq0iu5.cloudfront.net/ads1/data/lambda_virus.fa

Go to next item ✓ Completed

Like Dislike Report an issue

Go to next item →